

UML Modeling and Diagrams

1. Introduction
2. The class diagram
3. Object and sequence diagrams
4. State-transition diagram
5. The Use Case Diagram
6. Exercises



1

1 - Introduction



2

The UML language

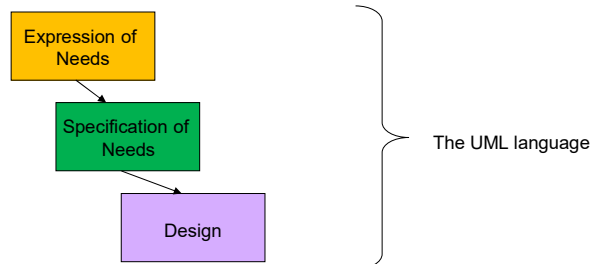


- UML: Unified Modeling language
- UML is:
 - a modeling language that is **easily understood** by humans and interpretable by machines,
 - a **visual language** that uses graphic notation,
 - in the **public domain**, it is a norm,
 - a language compatible with all production technologies.
- UML is **not** a **software development method**.

The UML language



- The UML language offers a set of concepts and notations
 - Grouped in different diagrams (UML 2.5.1, 13 diagrams)
 - Cover the different phases of software development



UML in practice



- In a study of 109 professionals in 34 countries, 88% of participants used UML to model their systems.
 - Primary use of class, sequence, deployment, and activity diagrams
 - Ozkaya, M., & Erata, F. (2020). A survey on the practical use of UML for different software architecture viewpoints. *Information and Software Technology*, 121, 106275. <https://doi.org/10.1016/j.infsof.2020.106275>

UML in practice



- Another study shows that diagrams and templates are mostly used as means of communication, thinking, and collaboration, rather than extremely formal documents that need to stay perfectly synchronized with the code.
 - Gorschek, T., Tempero, E., & Angelis, L. On the use of software design models in software development practice: An empirical investigation. *The Journal of Systems and Software*, 95, 176–193. <https://doi.org/10.1016/j.jss.2014.06.016>



• The main modeling tools:

Outil	Type	Modèle UML réel	Génération code	Reverse engineering	Gratuit / Open source	Facilité débutant
Enterprise Architect	Modèleur professionnel	★★★★★	★★★★★	★★★★★	✗ / académique	★★
Visual Paradigm	Modèleur professionnel	★★★★★	★★★★★	★★★★★	● Community	★★★★
StarUML	Modèleur développeur	★★★★	★★★★★	★★★★★	✗	★★★★
Modelio	Modèleur open source	★★★★	★★★	★★★	✓	★★★
PlantUML	Diagram-as-code	★★	—	—	✓	★★★
diagrams.net	Outil de dessin	★	—	—	✓	★★★★★

• StarUML

- Modern, relatively lightweight interface, easy to use
- Windows, MacOS, and Linux.
- UML 2.x, code generation, reverse engineering.
- Extension system.
- **Paid.**

Cours de génie logiciel - Michel Ianotto

7

7

2 - The class diagram



Cours de génie logiciel - Michel Ianotto

8

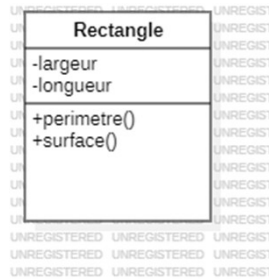
8

The class diagram



- Allows you to represent **classes** and **relationships** between classes

- Class**



- Relationship between classes**

- Association, aggregation, composition
- Generalization
- Dependency

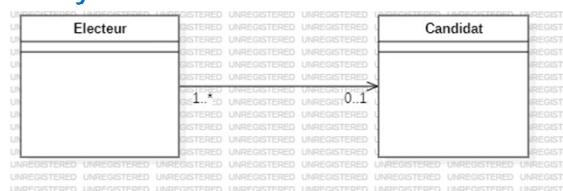
The class diagram



- Association** between classes



- Navigability** of an association





Two-way association of 1 to many between Person and Company

11

Personne class (1)



```
#pragma once
#include <string>
#include <memory>
class Entreprise;

class Personne {
private:
    std::string nom;
    double salaire;
    std::shared_ptr<Entreprise> employeur;

public:
    // Constructor
    Personne(const std::string& unNom, double unSalaire);

    // Initializes the referral to the employer
    void setEmployeur(std::shared_ptr<Entreprise> unEmployeur);
};
```

12

Personne class (2)



```
#include "Personne.hpp"

Personne::Personne(
    const std::string& unNom,
    double unSalaire)
    : nom(unNom),
      salaire(unSalaire),
      employeur(nullptr) {

}

void Personne::setEmployeur(
    std::shared_ptr<Entreprise> unEmployeur) {
    employeur = unEmployeur;
}
```

Entreprise class (1)



```
#pragma once

#include <string>
#include <vector>
#include <memory>

class Personne;

class Entreprise {
private:
    std::string nom;
    std::vector<std::shared_ptr<Personne>> employe;
public:
    // Constructor
    Entreprise(const std::string& unNom);

    // Adds an employee to the list
    void ajouteEmploye(std::shared_ptr<Personne> unEmploye);
};
```

Entreprise class (2)



```
#include "Entreprise.hpp"

Entreprise::Entreprise(const std::string& unNom)
    : nom(unNom) {}

void Entreprise::ajouteEmploye(
    std::shared_ptr<Personne> unEmploye) {
    employe.push_back(unEmploye);
}
```

The main function



```
#include "Personne.hpp"
#include "Entreprise.hpp"

#include <memory>

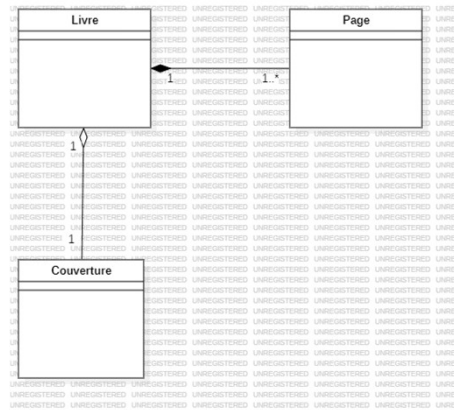
int main() {
    // Creating Objects
    auto unePersonne1 = std::make_shared<Personne>("P1", 10000);
    auto unePersonne2 = std::make_shared<Personne>("P2", 5000);
    auto uneEntreprise = std::make_shared<Entreprise>("E1");

    // Creating Links Between Objects
    unePersonne1->setEmployeur(uneEntreprise);
    uneEntreprise->ajouteEmploye(unePersonne1);
    unePersonne2->setEmployeur(uneEntreprise);
    uneEntreprise->ajouteEmploye(unePersonne2);

    return 0;
}
```

The class diagram

- Aggregation and composition between classes



Génie logiciel - Michel Ianotto

17

17

Unidirectional 1-to-1 dialing between Person and Civil Status

Cours de génie logiciel - Michel Ianotto

18

18

Personne class (1)



```
#pragma once

#include <string>
#include <memory>
#include <iostream>

class EtatCivil;

class Personne {
private:
    std::string nom;
    std::string adresse;
    std::unique_ptr<EtatCivil> unEtatCivil;
```

Personne class (2)



```
public:
    // Constructor
    Personne(
        const std::string& unNom,
        const std::string& uneAdresse,
        const std::string& uneDateNaissance,
        const std::string& unLieuNaissance);

    // Destructor
    ~Personne();

    // Operator overload <<
    friend std::ostream& operator<<(
        std::ostream& os,
        const Personne& personne);
};
```

Personne class (3)



```
#include "Personne.hpp"
#include "EtatCivil.hpp"

Personne::Personne(
    const std::string& unNom,
    const std::string& uneAdresse,
    const std::string& uneDateNaissance,
    const std::string& unLieuNaissance)
    : nom(unNom),
      adresse(uneAdresse),
      unEtatCivil(
          std::make_unique<EtatCivil>(
              uneDateNaissance,
              unLieuNaissance)) {}
```

Personne class (4)



```
Personne::~~Personne() {}

// Operator overload <<
std::ostream& operator<<(std::ostream& os, const Personne& personne) {
    os << "Nom : " << personne.nom << std::endl;
    os << "Adresse : " << personne.adresse << std::endl;
    os << *personne.unEtatCivil;

    return os;
}
```

EtatCivil class (1)



```
#pragma once

#include <string>
#include <iostream>

class EtatCivil {
private:
    std::string dateNaissance;
    std::string lieuNaissance;

public:
    // Constructor
    EtatCivil(
        const std::string& uneDateNaissance,
        const std::string& unLieuNaissance
    );
};
```

EtatCivil class (2)



```
// Operator overLoad <<
friend std::ostream& operator<< (
    std::ostream& os,
    const EtatCivil& etatCivil
);
};
```

EtatCivil class (3)



```
#include "EtatCivil.hpp"

EtatCivil::EtatCivil(
    const std::string& uneDateNaissance,
    const std::string& unLieuNaissance)
    : dateNaissance(uneDateNaissance),
      lieuNaissance(unLieuNaissance) {
}

// Operator overload <<
std::ostream& operator<<(
    std::ostream& os,
    const EtatCivil& etatCivil) {
    os << "date " << etatCivil.dateNaissance << std::endl;
    os << "lieu " << etatCivil.lieuNaissance;

    return os;
}
```

Cours de génie logiciel - Michel Ianotto

25

25

The main function (1)



```
#include "Personne.hpp"

#include <memory>
#include <iostream>

int main() {
    // Declaration and creation of objects
    auto personne1 = std::make_shared<Personne>(
        "Dupond",
        "Metz",
        "1989",
        "Paris"
    );
    auto personne2 = std::make_shared<Personne>(
        "Martin",
        "Nancy",
        "2001",
        "Nantes"
    );
}
```

Cours de génie logiciel - Michel Ianotto

26

26

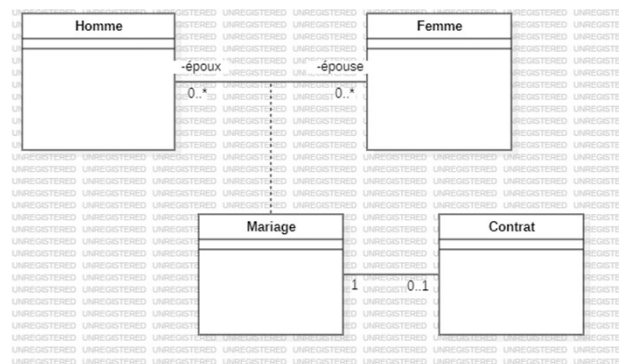
The main function (2)

```
// Viewing People Information
std::cout << *personne1 << std::endl;
std::cout << *personne2 << std::endl;

return 0;
}
```

The class diagram

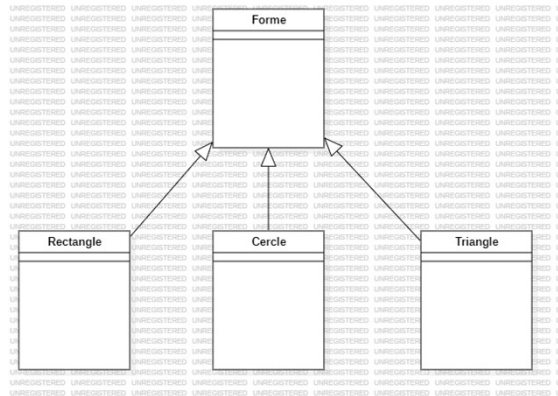
- Association classes



The class diagram



• The generalization relation



Génie logiciel - Michel Ianotto

29

29

The class diagram



- The relationship of dependency
 - Class A occasionally uses Class B
 - There is no long-term relationship between classes A and B



```

class Client {
public:
    void consulter(const Rapport& rapport);
};
  
```

Génie logiciel - Michel Ianotto

30

30



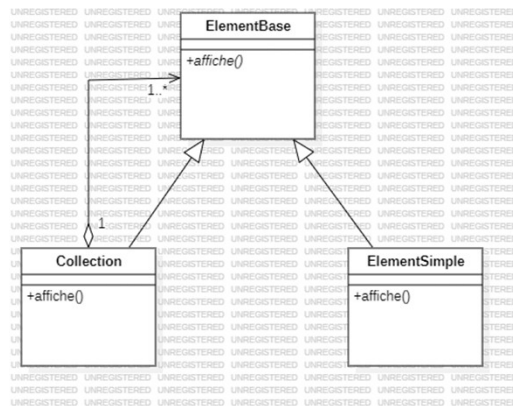
Recursive aggregation

31

The class diagram



- The class diagram of the recursive aggregation



32

ElementBase class (1)



```
#pragma once

#include <iostream>

class ElementBase {
public:
    virtual ~ElementBase() = default;

    virtual void afficher(std::ostream& os) const = 0;
};

std::ostream& operator<<(
    std::ostream& os,
    const ElementBase& element
);
```

ElementBase class (2)



```
#include "ElementBase.hpp"

std::ostream& operator<<(std::ostream& os, const ElementBase& element) {
    element.afficher(os);

    return os;
}
```

ElementSimple class (1)



```
#pragma once

#include "ElementBase.hpp"

class ElementSimple : public ElementBase {
private:
    int valeur;

public:
    // Constructor
    ElementSimple(int v);

    // Display
    void afficher(std::ostream& os) const override;
};
```

35

ElementSimple class (2)



```
#include "ElementSimple.hpp"

ElementSimple::ElementSimple(int v)
    : valeur(v) {
}

void ElementSimple::afficher(std::ostream& os) const {
    os << "Valeur " << valeur << std::endl;
}
```

36

Collection class (1)



```
#pragma once

#include "ElementBase.hpp"

#include <vector>
#include <memory>

class Collection : public ElementBase {
private:
    std::vector<std::shared_ptr<ElementBase>> lesElements;

public:
    // Adds an element
    void ajoute(std::shared_ptr<ElementBase> unElement);

    // Display
    void afficher(std::ostream& os) const override;
};
```

Cours de génie logiciel - Michel Ianotto

37

37

Collection class (2)



```
#include "Collection.hpp"

void Collection::ajoute(std::shared_ptr<ElementBase> unElement) {
    lesElements.push_back(unElement);
}

void Collection::afficher(std::ostream& os) const {
    for (const auto& element : lesElements) {
        os << *element;
    }
}
```

Cours de génie logiciel - Michel Ianotto

38

38

The main function (1)



```
#include "ElementSimple.hpp"
#include "Collection.hpp"

#include <iostream>
#include <memory>

int main() {
    // Creating Tree Elements
    auto e1 = std::make_shared<ElementSimple>(2);
    auto e2 = std::make_shared<ElementSimple>(5);

    auto c1 = std::make_shared<Collection>();
    auto c2 = std::make_shared<Collection>();
```

The main function (2)



```
auto e3 = std::make_shared<ElementSimple>(100);
auto e4 = std::make_shared<ElementSimple>(200);

// Construction of the n-ary shaft
c2->ajoute(e3);
c2->ajoute(e4);

c1->ajoute(c2);
c1->ajoute(e1);
c1->ajoute(e2);

// Display of the n-ary tree
std::cout << *c1;

return 0;
}
```

Practical advice



1. Class Naming and class hierarchy

- Choose the class names carefully
- Choose a descriptive name that helps understand the role of the class
- Choose a name that is not confusing
- Avoiding class hierarchies that are too deep

Practical advice



2. Reference

1. Do not introduce class names as an attribute
2. Model object references as associations

3. Diagram layout

1. Propose a simple model
2. Arrange the classes so that there are a minimum of intersections between the arcs that connect the classes

Practical advice



4. **Review** and documentation
 1. Ask others to review your model
 2. Document your model

3 - Object and sequence diagrams



CentraleSupélec



The Object Diagram

Cours de génie logiciel - Michel Ianotto

45

45

CentraleSupélec



Object diagram

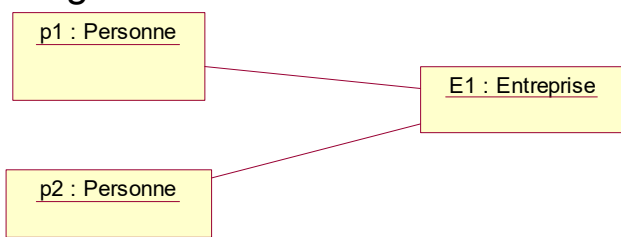
- Class diagram



```

classDiagram
    Personne "1..n" -- "1" Entreprise
  
```

- Object diagram = **instance** of the class diagram



```

classDiagram
    class "p1 : Personne"
    class "p2 : Personne"
    class "E1 : Entreprise"
    "p1 : Personne" -- "E1 : Entreprise"
    "p2 : Personne" -- "E1 : Entreprise"
  
```

Cours de génie logiciel - Michel Ianotto

46

46



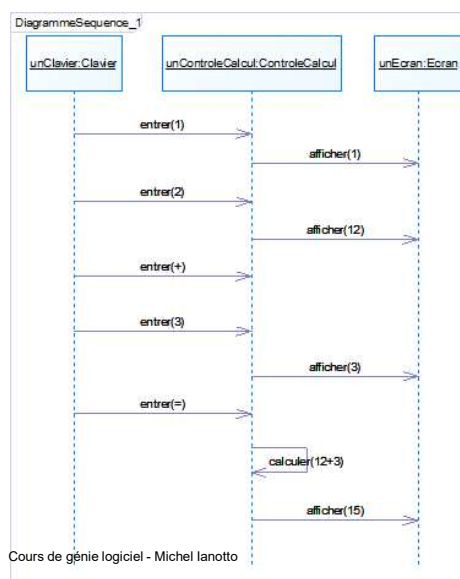
The sequence diagram

47

Sequence diagram



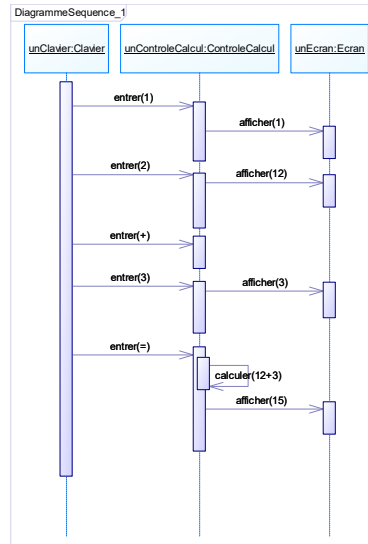
- A diagram that shows the messages exchanged between the Keyboard, Control, and Screen objects.



48

Sequence diagram

- Diagram shows that the messages exchanged between the Keyboard, ControlCalculation, and Screen objects.
- Presence of activity periods on the diagram.



Cours de génie logiciel - Michel Ianotto

49

49

4 - The transition state diagram

Cours de génie logiciel - Michel Ianotto

50

50

The transition state diagram



- Class diagram



Génie logiciel - Michel Ianotto

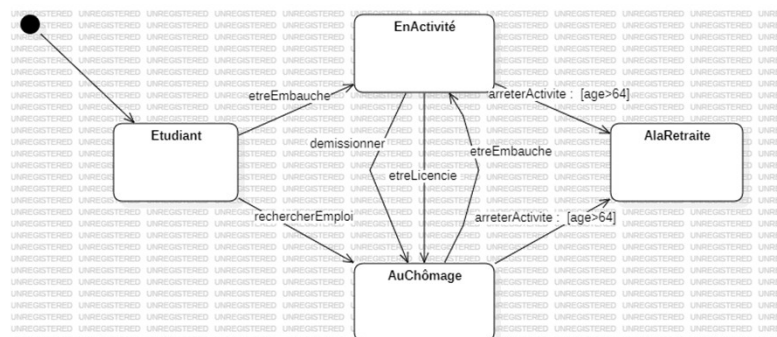
51

51

The transition state diagram



- Les états d'un objet = instance de la classe `Personne`



Génie logiciel - Michel Ianotto

52

52

The transition state diagram

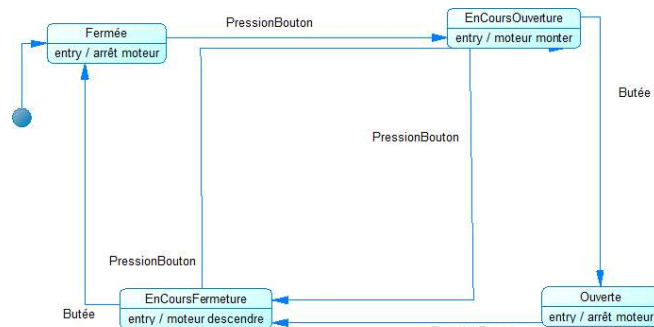


- A motorized **garage door** is considered. The user has a remote control with a single button to operate the door. Pressing the button has the effect of:
 - **open** the door if it is closed,
 - to **close** it if it is open,
 - to **request its closure** if it is in the process of opening,
 - to **request its opening** if it is in the process of being closed.

The transition state diagram



- The door has a stop sensor that indicates that the door has reached its high (**door open**) or low (**door closed**) stop.



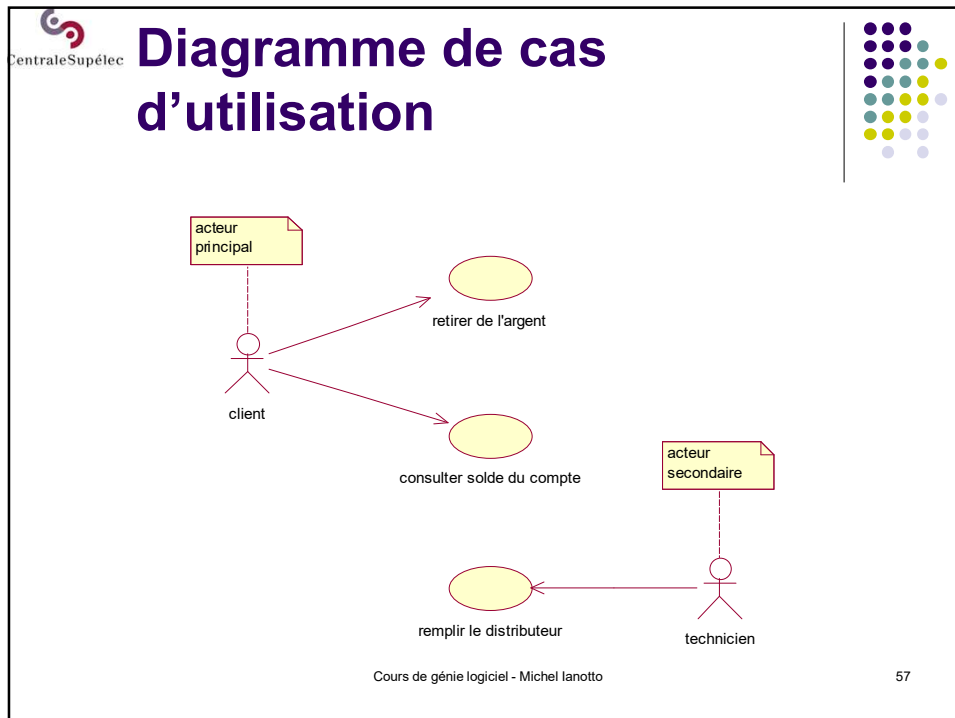
5 - The diagram of use cases



Définition



- Describes what the system should do
- Used during the requirements specification phase
 - Allows you to delineate the boundaries of the system
- Modeling elements
 - Main and secondary **actors**
 - **Use cases**
 - **Relationships** between actors and use cases or between use cases



57

CentraleSupélec

6 - Exercices

Cours de génie logiciel - Michel Ianotto

58

58

Exercice n°1



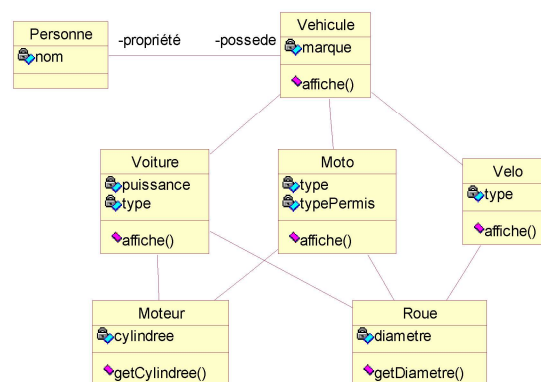
Give the class diagram for the following situations:

1. People associated with the university are students and professors
2. A rectangle is characterized by the coordinates (x, y) of the upper left corner and the lower right corner. A rectangle is constructed from these 2 coordinates. It is possible to calculate its area and perimeter and to translate it.

Exercice n°2 (1)



- The following class diagram is given::



Exercice n°2 (2)



1. Complete the relations in this diagram, indicate their multiplicity.
2. Give the C++ declaration of the Person, Vehicle, Car and Engine classes. For each of these classes, the type of attributes and the header of the methods will be indicated.
3. Give the definition of the poster method of the Car class which must display the make, power and type of the car as well as the engine displacement and the diameter of the wheels.

Exercice n°3



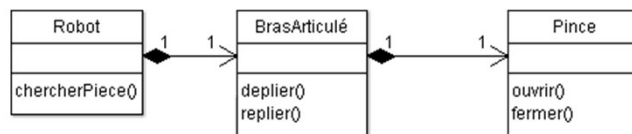
A drawing is made up of simple figures and compound figures. A composite figure consists of several figures (simple and compound) and a simple figure can be a point, a line, or a circle. A figure can be drawn or translateral.

1. Give the modeling of the classes of this situation

Exercice n°4



- The class diagram below models a robot that has an articulated arm ending in a gripper. The robot works as follows: the robot unfolds its arm, grabs the workpiece with its gripper, folds its arm and then releases the part.



- Use a sequence diagram to represent the exchange of messages between robot, articulated arm and gripper objects.

Exercice n°5



The sequence diagram on the next page describes the scenario of identifying an individual to a system.

- The periods of activity of the objects given on the sequence diagram are not exact. Correctly indicate the period of activity of each object.
- Give the class diagram.

